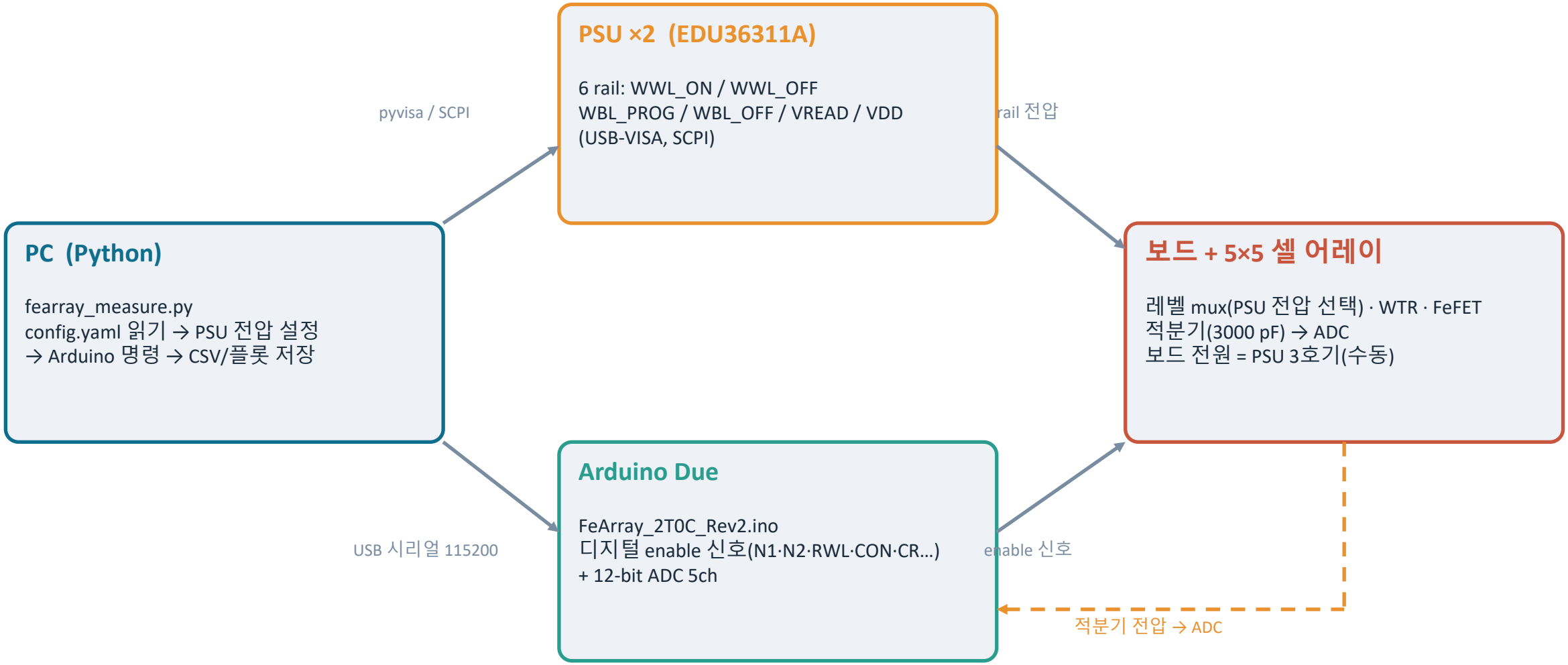
2T0C FeFET 5×5 어레이 측정 셋업 설명서

동작 원리 · 연결 · 전압 운용 · 펄스/시나리오 · GUI 사용법 · 오실로스코프 검증 (Rev2, 2026-10)

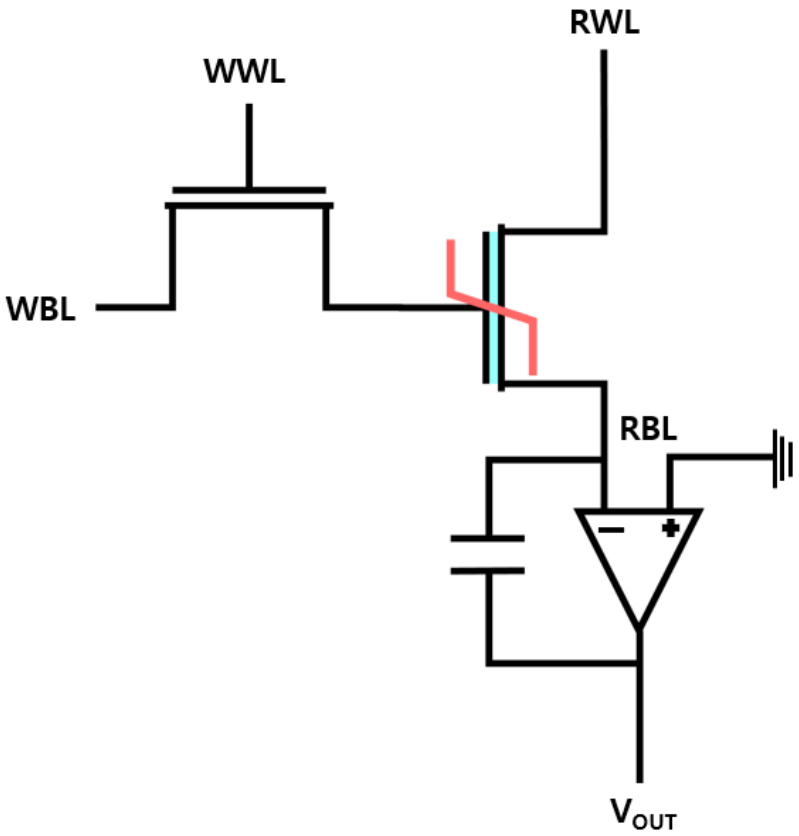
nmdl · Rev2 매뉴얼 (2판)

전체 구성: PC → PSU(전압) + Arduino(타이밍) → 보드 → ADC



원칙: 전압 크기는 PSU가, 언제·얼마나 오래 인가할지는 Arduino가 정한다. Arduino 핀은 3.3 V 로직이며 보드의 mux/레벨시프터가 PSU rail을 셀에 연결한다.

2T0C 셀: WTR + FeFET, 그리고 적분기 읽기 회로



Program (P)

WWL = WWL_ON → WTR 켜짐
WBL = WBL_PROG → SN(FeFET 게이트)에 V_prog
→ HZO 분극 switching, FeFET Vth ↓ (전류 ↑)

Erase (D)

WBL = WBL_OFF(≈0 V) 상태에서 WWL = WWL_ON
→ SN이 0 V로 → 반대 방향 분극, Vth ↑ (전류 ↓)

Read (R)

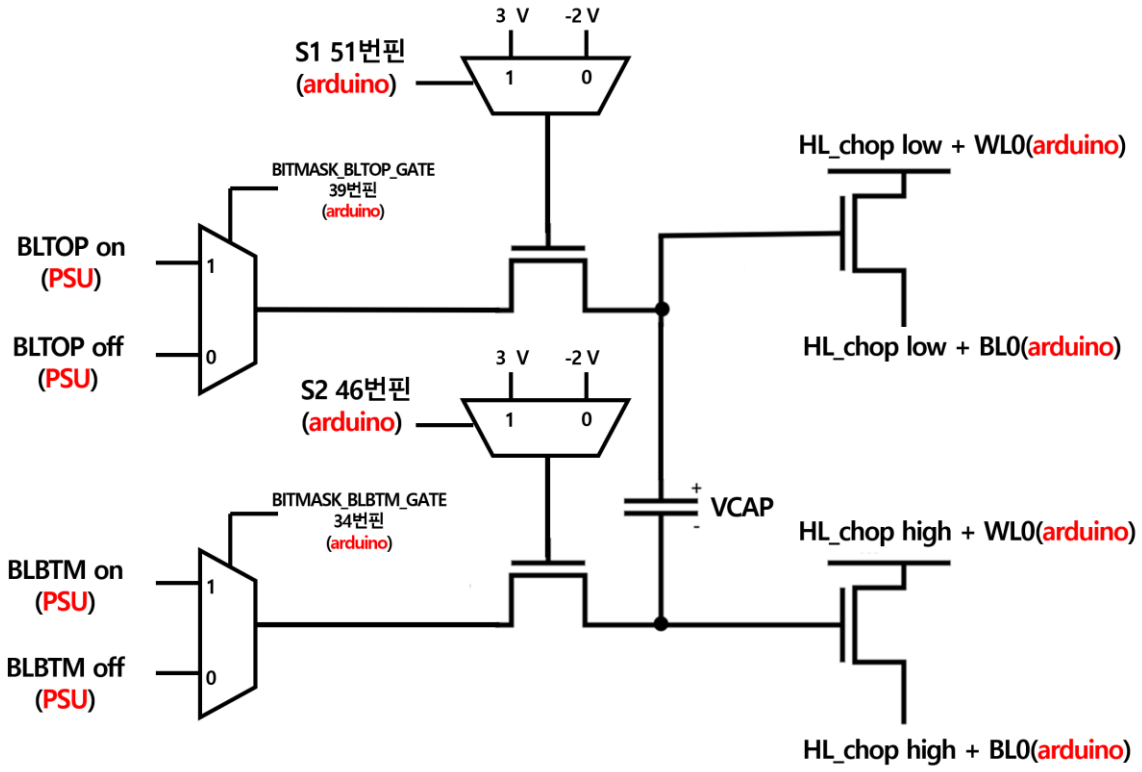
WWL = WWL_OFF → WTR 꺼짐, SN 플로팅 (저장 유지)
RWL 펄스 → FeFET 전류 I_RBL → 적분기 cap에 축적
$V_{OUT} = -(1/C)\int I_{RBL}\,dt$ → ADC (비파괴 읽기)

Hold / Retention

WWL = WWL_OFF, WBL = WBL_OFF, RWL off
FeFET 분극이 상태를 보존 → 리프레시 불필요

네 신호선: WWL/WBL = 쓰기 (Arduino N1/N2 → PSU rail 선택), RWL/RBL = 읽기 (Arduino RWL 펄스 → 적분기 → ADC). SN = 저장 노드 = FeFET 게이트

보드 구현: 셀 신호를 PSU rail과 Arduino 핀으로 만드는 방법



보드 회로도 (row 0 / col 0 셀 기준)

2T0C 신호	보드 회로도 위치	만들어지는 방법
WWL	S1 → 상단 pass TR gate	Arduino N1 HIGH→WWL_ON rail, LOW→WWL_OFF rail
WBL	BLTOP mux 출력	Arduino N2 HIGH→WBL_PROG, LOW→WBL_OFF
SN	VCAP 노드	상단 pass TR(=WTR) 아래 노드 = FeFET 게이트
RWL	상단 read TR gate (HL_chop LOW)	Arduino RWL[row] 펄스 (DFF 래치)
RBL	read TR 드레인 → 적분기	op-amp + 3000 pF → ADC[col]
S2 / BLBTM	하단 경로	4T0C용, 2T0C에서는 미사용 (S2 LOW)

회로도의 3 V / −2 V 자리에 WWL_ON / WWL_OFF rail이, BLTOP on/off 자리에 WBL_PROG / WBL_OFF rail이 들어간다. 값은 모두 PSU(yaml voltages)로 바꾼다.

연결도

Arduino 핀 ↔ 신호 ↔ 셀 좌표, PSU 2대 6채널 ↔ 전압 rail

Arduino Due 핀 매핑 (펌웨어 PIN 테이블 = 4T1C 펌웨어와 동일)

신호	선택 인덱스	핀 (idx 0→4)	포트 비트	비고
N1 = WWL	col	51, 50, 49, 48, 47	PC12…PC16	회로도 S1 = 51번
N2 = WBL 레벨	row	39, 38, 37, 36, 35	PC7…PC3	회로도 BLTOP gate = 39번
RWL	row	25, 26, 27, 28, 29	PD0,1,2,3,6	DFF(CK=31번)로 래치
RBL → ADC	col	A0, A1, A2, A3, A4	ADC ch7…ch3	raw 12-bit /4 → 0~1023
CON / DS / CR	—	53 / 42 / 52	PB14 / PA19 / PB21	적분기 연결 / 샘플 / cap 리셋
HL_CHOP / FB	—	2 / 22	PB25 / PB26	항상 LOW (상단 read TR)

Arduino 핀 출력은 0/3.3 V 로직. 실제 셀 전압은 보드의 mux가 PSU rail 중 하나를 고르는 방식 (HIGH→ON rail, LOW→OFF rail).

셀 좌표 규약 (Rev1.0 계승)

- cell (r, c): 쓰기 = N1[c] & N2[r], 읽기 = RWL[r] & ADC[c]
- row_mask × col_mask의 교차곱이 선택됨
- 예) row [0,1] × col [0,1] → 4셀 동시

꼭 확인

- WWL이 col, WBL이 row로 가는 규약이 PCB 배선과 맞는지 확인
- 다르면 .ino 상단 N1_PIN / N2_PIN / WL_PIN / ADC_CH 테이블만 수정
- INIT → DBGR로 25셀 맵이 정상 범위(수십~수백)인지 먼저 확인

PSU 연결: EDU36311A 2대 → 6 rail (별칭 POWER_SUPPLY_1/2)

rail	PSU	CH	최대	극성	보드 연결 (mux 입력)	예시 값
VDD	1	CH1	6 V	+	보드 아날로그 전원 (적분기/레벨시프터)	5.0 V
WWL_ON	1	CH2	30 V	+	S1 mux '1' 입력 (N1 HIGH)	3.0 V
WWL_OFF	1	CH3	30 V	– (반전 배선)	S1 mux '0' 입력 (N1 LOW)	−2.0 V
VREAD	2	CH1	6 V	– (반전 배선)	RWL read 바이어스 — 가상접지(VINP)보다 낮게	−1.0 V
WBL_PROG	2	CH2	30 V	+	BLTOP 'on' 입력 (N2 HIGH)	4.0 V
WBL_OFF	2	CH3	30 V	– (반전 배선)	BLTOP 'off' 입력 (N2 LOW) = erase 레벨	0 ~ −3 V

PSU 1 (EDU36311A)

CH1 VDD
+ → VDD, − → GND

CH2 WWL_ON
+ → S1 mux '1', − → GND

CH3 WWL_OFF
− → S1 mux '0', + → GND (반전)

PSU 2 (EDU36311A)

CH1 VREAD
− → VREAD, + → GND (반전)

CH2 WBL_PROG
+ → BLTOP 'on', − → GND

CH3 WBL_OFF
− → BLTOP 'off', + → GND (반전)

보드

- GND 한 점 공통 (PSU 1·2·3 모두)
- S1 mux 1/0 ← WWL_ON / WWL_OFF
- BLTOP on/off ← WBL_PROG / WBL_OFF
- VREAD, VDD, 보드 전원(PSU 3)
- Arduino GND ↔ 보드 GND

GUI는 NI-VISA 별칭(POWER_SUPPLY_1/2)으로 자동 배정. PSU3(POWER_SUPPLY_3)은 보드 전원, 코드가 건드리지 않음. 켜는 순서 on_order: VDD → WWL_OFF → WBL_OFF → VREAD → WWL_ON → WBL_PROG / 끝나면 off_order 역순으로 3단계 램프다운 후 출력 OFF

음전압 rail 만들기

- EDU36311A 출력은 절연 → 채널 '+' 단자를 보드 GND에, '−' 단자를 rail로 연결
- yaml에서 negative: true 로 표시 (드라이버는 |V|만 설정)
- 음전압 요청인데 negative:false면 에러로 막음

배선 규칙

- 모든 PSU의 '−' (또는 반전 시 '+') 기준을 보드 GND 한 점에 공통 접지
- CH1은 6 V 제한 → 큰 전압은 CH2/CH3
- 보드용 PSU 3호기는 코드가 건드리지 않음 (먼저 켜고 마지막에 끔)
- VISA 주소: pyvisa list_resources()로 확인 후 yaml 입력

전압 운용 원칙: WBL 세 레벨과 RWL 극성

WBL에는 세 레벨이 필요하다

- Potentiation(+): program 펄스 때 게이트에 걸리는 전압 → WBL_PROG rail (N2 HIGH)
- Rest/Read(0 V): program 펄스 끝에 게이트가 돌아오는 레벨, 읽기 중 WBL 레벨 → WBL_OFF rail (N2 LOW)
- Depression(−): erase 펄스 때 게이트에 걸리는 전압 → 역시 WBL_OFF rail (N2 LOW + N1 펄스)
- → mux 입력이 둘뿐이라 WBL_OFF가 0 V와 −V 두 역할을 겸함

해법: phase마다 WBL_OFF를 바꾼다 (phase_voltages)

- P phase: WBL_OFF = 0 V → 펄스 끝에 게이트 0 V, 안전
- D phase: WBL_OFF = −3 V → erase 펄스가 −3 V를 걸고, 펄스 사이엔 N1 OFF(플로팅)
- Python이 set마다 PSU를 바꿈(≈0.5 s). GUI: 'P/D phase에 다른 전압' 체크 → D열 WBL_OFF=−3
- 이 방식에서는 read_mode = N1OFF (N1ALL이면 D 구간에 게이트가 −3 V에 묶임)

RWL(VREAD)은 음수여야 한다

- 적분기는 반전형: V_OUT = −(1/C)∫I dt. 전류가 가상접지로 '들어오면' V_OUT이 음 → ADC 바닥(≈12 counts)
- ADC가 올라가려면 전류가 가상접지에서 셀로 '나가야' → RWL < VINP(가상접지) → VREAD 음수 (예 −1 V)
- FeFET은 RWL이 낮은 쪽이 source가 되어 Vgs = V_SN − V_RWL > 0 → 전류 흐름. Vth 차이 그대로 반영
- VINP가 0 V가 아닐 수 있음 → 보드 설정 확인. 검증: VREAD 0 → −0.5 → −1 V로 내리며 ADC 상승 확인

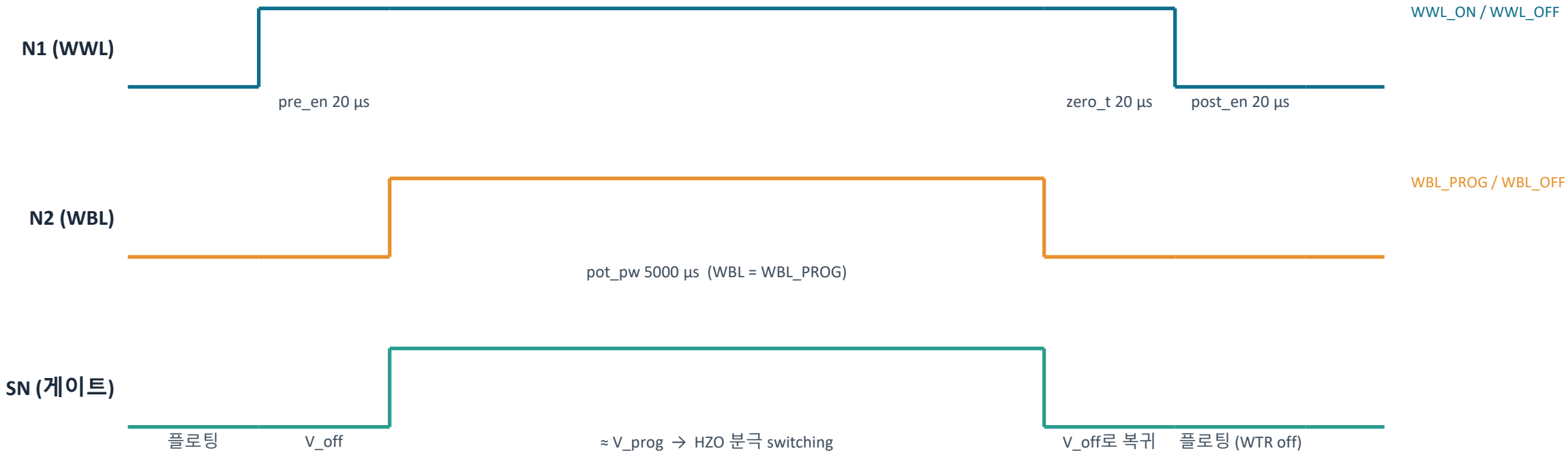
음전압 rail 만들기 / 안전

- EDU36311A는 양극 출력만 → 채널 + 단자를 보드 GND에, − 단자를 rail에 (반전 배선)
- 반전 rail: WWL_OFF, WBL_OFF, VREAD. yaml negative:true, 코드는 |V|만 설정하고 극성 불일치는 에러
- 측정 중 단자를 바꿀 일은 없음: 0 V와 −V는 같은 반전 배선으로 모두 가능
- 하단 경로(S2/BLBTM)를 depression 전용으로 쓰는 3-rail 방식은 BLBTM 선택 핀 확인 후 추가 예정

펄스는 어떻게 나가는가

Program / Erase / Read 타이밍과 현재 기본 조건

Program 펄스 (scenario P, 셀당 1회)



펌웨어 동작 (programPulse)

- N1[col] ON → pre_en → N2[row] ON → pot_pw → N2 OFF → zero_t → N1 OFF → post_en
- wwl_hold=1 이면 N1을 program phase 내내 ON (Rev1.0 FE 모드와 동일) — N2만 펄스

조절

- 진폭: yaml voltages.WBL_PROG (PSU) — 펄스마다 바꾸려면 sweep
- 폭: pot_pw [µs] · 횟수: pot_num · WTR 게이트 전압: WWL_ON
- 게이트가 V_off로 돌아올 시간: zero_t (WTR RC보다 길게)

Erase 펄스 (scenario D, 셀당 1회)



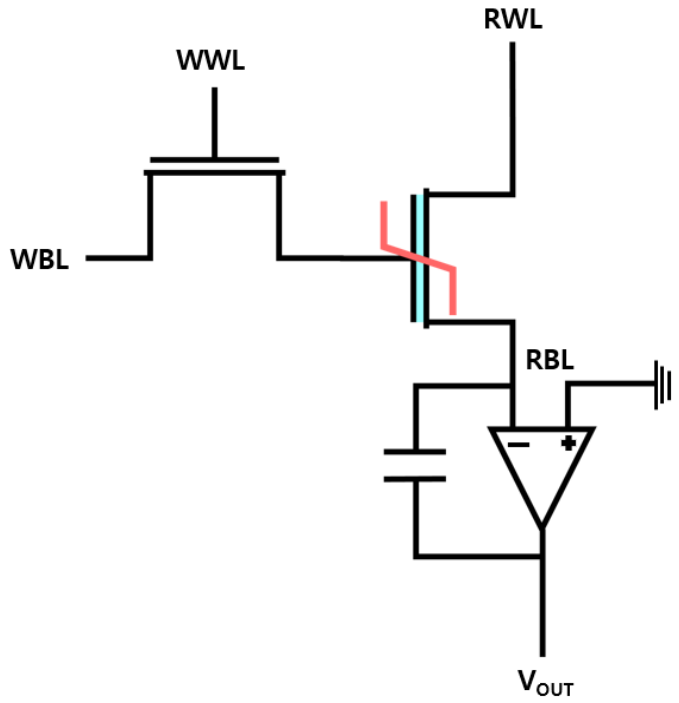
펌웨어 동작 (erasePulse)

- N2 전부 OFF → pre_en → N1[col] ON → dep_pw → N1 OFF → post_en → zero_t
- Erase는 'WBL=V_off인 채로 WTR을 켜는 것' → 같은 col의 다른 row 셀도 같은 조건에 놓임 (half-select)

조절 / 주의

- erase 강도 = WBL_OFF 전압(음수 가능, negative 배선) × dep_pw × dep_num
- Erase 중 WWL_ON이 WTR 게이트에 걸림 → GI 스트레스. 필요 시 WWL_ON을 낮게
- disturb 평가의 핵심 구간: 같은 col 비선택 row가 함께 erase되는지 맵에서 확인

Read 동작: FeFET 전류를 적분해서 읽는다 (비파괴)



1 준비

WTR 꺼짐(WWL_OFF) → SN 플로팅, 저장 상태 유지. CON LOW로 RBL을 적분기에 연결, CR HIGH로 cap 개방

2 적분

RWL[row]을 read_time × 100 µs 동안 HIGH (DFF 래치). FeFET이 Vth에 따라 I_RBL을 흘리고 cap에 Q = ∫I dt 축적. RBL은 op-amp 가상접지(≈0 V) → 셀 드레인 전압이 일정

3 샘플

RWL LOW, read_delay 후 CON HIGH로 분리 → 5개 col의 적분기 출력을 ADC가 동시에 읽음 (avg_count회 평균)

4 리셋

CR LOW로 cap 방전 3 ms → 다음 row. row 0→4 반복하면 25셀 맵 1회

값의 의미

- ADC(0~1023) ∝ V_cap = (1/C)·∫I_RBL dt
- Program 후 Vth ↓ → 전류 ↑ → ADC ↑ / Erase 후 ADC ↓

읽기 조건 (PSU + yaml)

- VREAD = RWL high 레벨 (read TR / FeFET 드레인 바이어스), WWL_OFF = read 중 WTR 차단 전압 (충분히 음수)
- read_time ↑ 감도 ↑ (포화 주의) · avg_count ↑ 노이즈 ↓ · read_mode로 read 중 WWL 상태 선택 (N1OFF 기본)

Read: 25셀 맵 (row 0→4 순서, row당 5 col 동시)



read_mode: read 중 WWL 상태

- N1OFF WTR off, 게이트 플로팅 (논문 방식, 기본)
- N1ALL 5 col WWL 모두 ON → 게이트 = WBL_OFF (전압 정의됨, 전 셀 동일 조건)
- N1SEL 선택 col만 ON

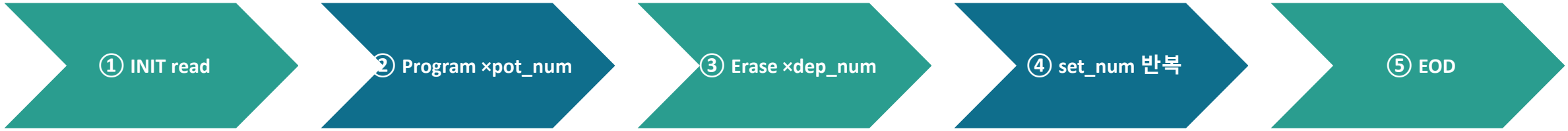
전류 역산 (Python)

- V_cap = ADC·4/4095·3.3 V, t = read_time·100 µs
- I = C·V_cap/t, C = 3000 pF → 기본 조건 1 count ≈ 0.1 nA
- ADC 포화(1023) 시 read_time을 줄일 것

조절

- read_time ↑ → 감도 ↑, 시간 ↑, 포화 위험
- avg_count ↑ → ADC 노이즈 ↓ (cap 충전은 1회)
- VREAD → read 바이어스 (PSU)

PD 시나리오 흐름과 현재 기본 조건



파라미터	기본값	의미	파라미터	기본값	의미
pot_num / dep_num	20 / 20	set당 P / D 펄스 수	read_time	1000	RWL 펄스 수 (×100 µs = 100 ms)
pot_pw / dep_pw	5000 / 5000 µs	펄스 폭	read_interval	1	N펄스마다 맵 read
pre_en / post_en / zero_t	20 / 20 / 20 µs	라인 간 settle	read_mode	N1OFF	read 중 WWL 상태
set_num	1~3	P→D 반복 횟수	avg_count	10	ADC 평균 횟수
WBL_PROG / WBL_OFF	4.0 / 0.0 V	program / erase 레벨 (PSU)	wwl_hold	0	P phase 동안 WWL 유지 여부
WWL_ON / WWL_OFF	3.0 / −2.0 V	WTR 게이트 (PSU)	read_wait_ms	100	read 전 대기

P와 D에 다른 전압: experiment에 phase_voltages: {P: {...}, D: {...}} 를 쓰면 Python이 set마다 PSU rail을 바꿔가며 P→D를 돌리고 한 폴더에 저장. 펌웨어 단독 PD는 같은 전압, phase_gap_ms로 P→D 사이 대기.

시간 예산: PD 기본(20P+20D, 매 펄스 read, read_time 1000)은 read 1회 ≈ 5 row × (100 ms + 3 ms) + 100 ms ≈ 0.6 s → 41 read ≈ 25 s/set. 펄스 자체(5 ms)는 무시 가능.

시나리오 한눈에 보기 (GUI '시나리오' 드롭다운 / yaml scenario)

시나리오	하는 일	출력	언제 쓰나
INIT	라인 전부 OFF 후 25셀 맵 1회 read	I 1회	측정 전 기준 맵, ADC 범위 확인
P	초기 맵 → 선택 셀에 program 펄스 pot_num회, read_interval마다 맵	I + P…	potentiation 응답만
D	초기 맵 → 선택 col에 erase 펄스 dep_num회, read_interval마다 맵	I + D…	depression 응답만
PD / DP	초기 맵 → (P phase → D phase) × set_num / DP는 순서 반대	I + P… D… 반복	identical pulse P/D 응답 + 비선택 셀 disturb
PD + phase_voltages	set마다 Python이 PSU를 P용/D용으로 바꿔가며 P·D phase 실행	같음 (한 폴더)	WBL_OFF를 0 V / −3 V로 나눠 써야 할 때 (권장)
R	맵을 r_points회, r_interval_s 간격으로 반복 read (펄스 없음)	R…	retention / 드리프트
DBGR	맵 1회 read	R 1회	수동 확인
DBGP / DBGD	program / erase 펄스 pot_num·dep_num회, read 없음	없음	펄스 1~2회 넣고 DBGR로 확인
LOOPP / LOOPD / LOOPR	program 펄스 1회 / erase 펄스 1회 / row 1개 read 를 ■중지('!')까지 무한 반복	LOOP> 만	오실로스코프 파형 확인

공통: 셀 선택은 row×col 교차곱. read 1회 = row 0→4 스윕(각 5 col 동시) = 25 셀 맵 5줄 'set,phase,pulse,row,ADC0..4>'. 모든 시나리오는 EOD>로 끝나고, 호스트가 '!'를 보내면 ABORT> EOD>.

무엇을 바꾸면 무엇이 달라지나

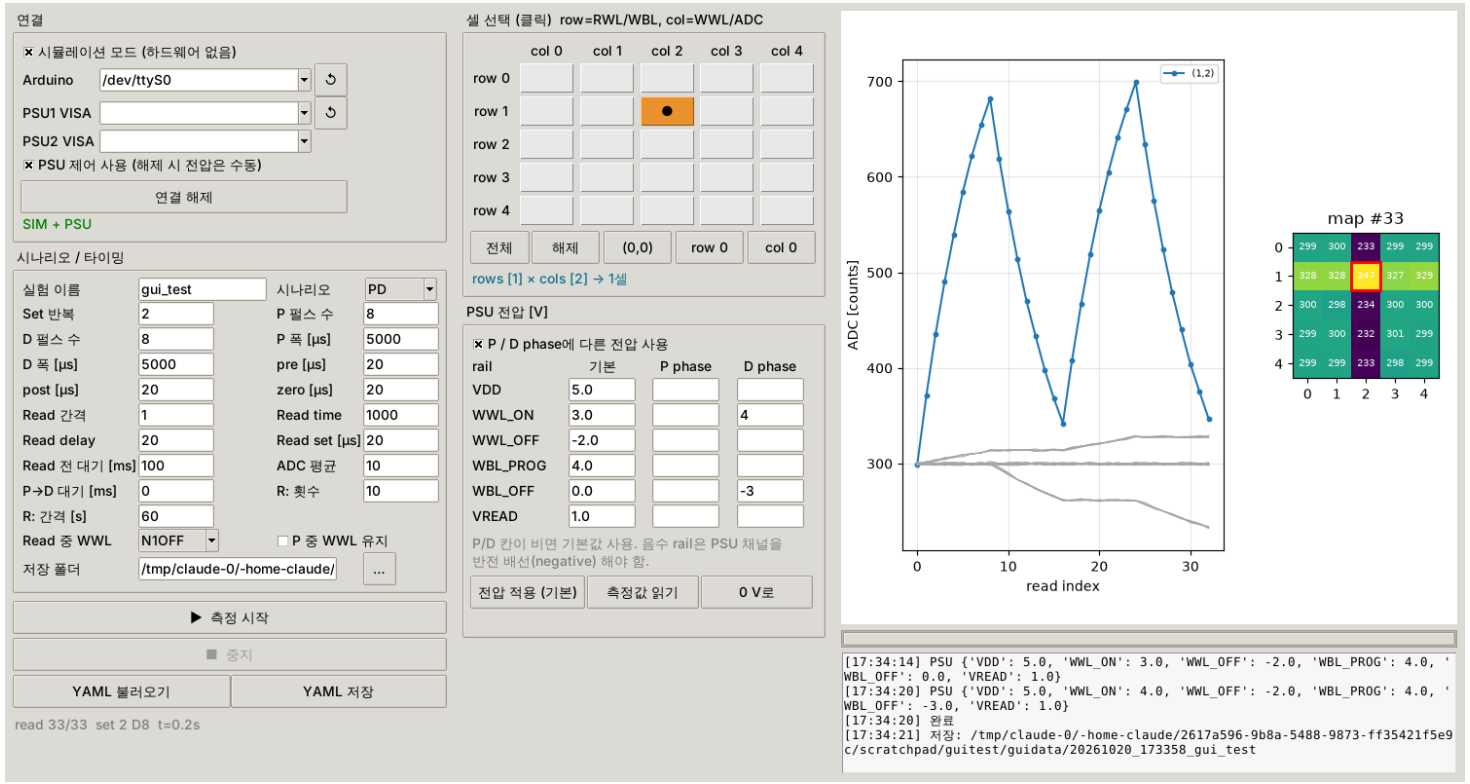
바꾸고 싶은 것	어디서	키	비고
Program 세기 (진폭)	PSU · yaml voltages	WBL_PROG	ISPP: experiments[].sweep {key: WBL_PROG, values: [...]}
Program 세기 (폭 / 횟수)	yaml defaults / experiment	pot_pw, pot_num	identical pulse = 폭·진폭 고정, 횟수만 누적
Erase 세기	PSU + yaml	WBL_OFF (음수 가능), dep_pw, dep_num	WBL_OFF는 read-write 레벨도 겸함 → 보통 0 V 유지, 폭·횟수로 조절
WTR 켜짐/꺼짐 강도	PSU	WWL_ON, WWL_OFF	WWL_OFF가 충분히 음수여야 게이트 플로팅 유지 (retention)
선택 셀	experiment	rows, cols	교차곱. 단일 셀 / 한 row / 한 col 단위 권장
read 빈도 / 분해능	experiment	read_interval, read_time, avg_count	read_interval 0 = 펄스만 (DBG)
read 중 게이트 조건	experiment	read_mode, wwl_hold	N1OFF↔N1ALL 비교로 read disturb 확인
P / D 전압 따로	experiment	phase_voltages: {P:{…}, D:{…}}	Python이 phase마다 rail 변경, 한 폴더에 저장. 음수 rail은 negative 배선 필요
Retention	scenario R	r_points, r_interval_s	P/D 후 맵을 주기적으로 반복 read

펌웨어 재업로드가 필요한 변경: 핀 테이블, RWL_PERIOD_US(100 µs), CAP_CLEAR_MS, 새 시나리오 추가. 나머지는 전부 yaml 한 줄.

사용법

GUI 순서 · 결과 파일 · 저장소

GUI (fearray_gui.py) 사용 순서



1 실행

Windows CMD: pushd \\wsl.localhost\Ubuntu-22.04\home\yunho\2t0c-array-setup\python → python fearray_gui.py (장치는 Windows만 보임)

2 연결

Arduino COM5, PSU1/PSU2는 ↺ → POWER_SUPPLY_1/_2 자동. '연결' → 녹색 'COM5 + PSU'. PSU는 0 V·출력 ON 상태

3 셀·전압

격자 클릭(교차곱 주의, + 표시). 전압 '기본' 열 입력, P/D 분리 시 체크 후 D열 WBL_OFF=−3

4 시나리오

드롭다운 선택, 펄스 수·폭·read 값 입력. '파형 미리보기' 탭에서 타이밍 확인

5 실행·저장

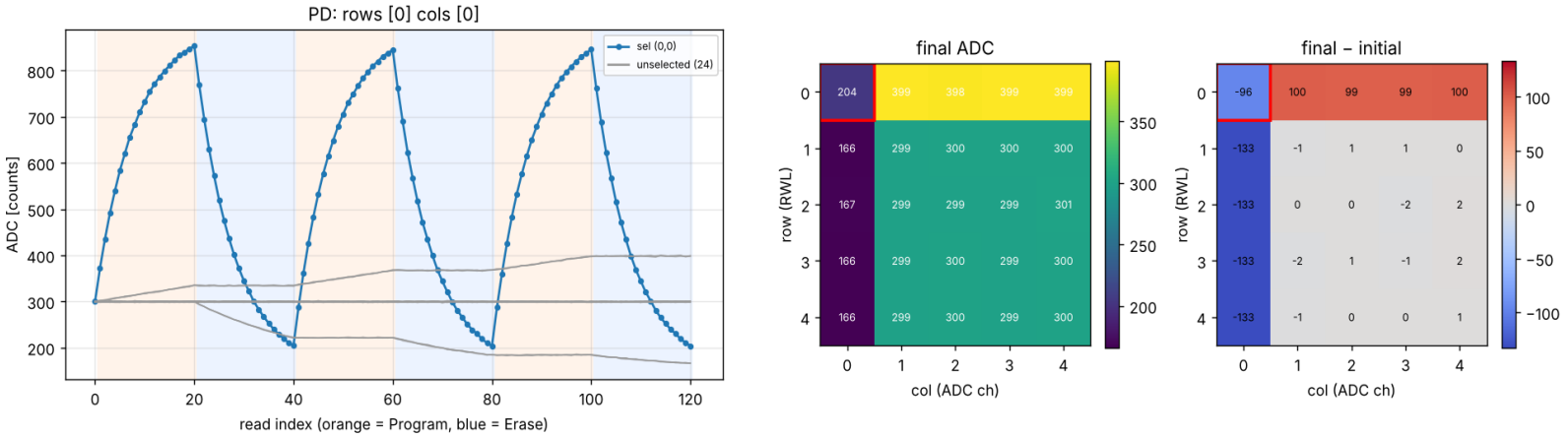
▶ 시작 → 실시간/셀별(5×5) 탭. ■ 중지는 현재 read 후 멈춤. 결과는 data/<시각>_<이름>/ (csv·맵·png·meta)

YAML 저장/불러오기로 배치 실행(fearray_measure.py config.yaml)과 호환. 시뮬레이션 체크 = 하드웨어 없이 전체 흐름 확인. 한글 글꼴이 없는 환경(conda Tk)은 영문 UI로 자동 전환.

환경

- 코드 편집·git: WSL(Ubuntu-22.04, VS Code Sync로 pull). 실행: Windows 파이썬 (COM·VISA·NI MAX 별칭은 Windows에서만 보임). 저장소: github.com/yunho-song/2t0c-array-setup

결과 파일과 플롯 읽는 법



plot.png (시뮬레이터 예시 — half-select disturb를 일부러 넣은 모델, 실제 소자 특성 아님)

data/<시각>_<이름>/

- data.csv
  read_idx,t_s,set,phase,pulse,row,col,selected,adc,I_uA
- map_initial / map_final / map_delta.csv
  5×5
- meta.json 파라미터·rail 전압·상수
- plot.png 아래 설명

왼쪽: 추이

- 색 선 = 선택 셀, 회색 = 비선택 24셀
- 주황 배경 = Program 구간, 파랑 = Erase 구간
- x = read 인덱스 (set·phase·pulse는 CSV에)

가운데/오른쪽: 맵

- final ADC 맵과 final − initial 맵, 빨간 테두리 = 선택 셀
- 선택 셀만 크게 변하고 나머지 Δ≈0 이면 OK
- 같은 col 또는 같은 row가 함께 움직이면 half-select disturb

콘솔 요약

- Δ selected: 선택 셀 평균 변화
- Δ unselected: 비선택 평균 / max|Δ|
- 실행 중 read 진행률과 선택 셀 값 표시

코드 저장소: github.com/yunho-song/2t0c-array-setup

```
2t0c-array-setup/
├─ README.md                          전체 설명 (이 PPT 요약)
├─ FeArray_2T0C_manual.pptx           이 설명서
├─ SCOPE_CHECK.md                     오실로스코프 검증 절차 (이 PPT 마지막 장 상세판)
├─ firmware/
│   └─ FeArray_2T0C_Rev2/
│       └─ FeArray_2T0C_Rev2.ino      Arduino Due 펌웨어
├─ python/
│   ├─ fearray_gui.py                 GUI (연결·전압·셀 선택·실시간/셀별 그래프·파형 미리보기)
│   ├─ fearray_measure.py             배치 실행 (yaml → PSU → Arduino → CSV/플롯)
│   ├─ fearray_arduino.py             시리얼 프로토콜 · ScenarioParams · 전류 역산
│   ├─ psu_macro_2T0C.py              6 rail ↔ PSU 채널 매핑 · 램프다운
│   ├─ power_supply_EDU36311A.py      SCPI 드라이버 (+Dummy)
│   ├─ fearray_sim.py                 하드웨어 없는 시뮬레이터 (--sim)
│   └─ config_example.yaml            설정 예시 → 복사해서 config.yaml
└─ data/                              측정 결과 (.gitignore, 커밋 안 함)
```

측정 데이터(data/)는 용량이 커지므로 저장소에 넣지 않고 OneDrive/연구노트 폴더에 둔다. 조건은 meta.json에 함께 저장되므로 커밋 해시만 기록하면 재현 가능.

처음 받기

- git clone https://github.com/yunho-song/2t0c-array-setup.git
- cd 2t0c-array-setup/python && pip install -r requirements.txt
- cp config_example.yaml config.yaml → 포트·VISA·전압 수정

바꾼 뒤 올리기

- git add -A && git commit -m "pot_pw 2ms, cell(1,2) PD"
- git push
- 펌웨어·yaml·측정 조건을 한 커밋으로 묶어 두면 데이터 추적이 쉬움

소자 연결 전 오실로스코프 검증

1단계 Arduino 단독 → 2단계 보드+PSU 낮은 전압 → 3단계 소자 삽입 후 read (상세: SCOPE_CHECK.md)

1단계: Arduino 단독 — 핀에서 타이밍이 맞게 나오는가

시나리오	프로브 (Due 핀)	기대 파형 (기본값)	확인
LOOPP, 셀(0,0)	51 = N1(WWL) / 39 = N2(WBL)	N1 High 5040 µs. N2는 20 µs 늦게 올라가 5000 µs, N1보다 20 µs 먼저 내려감	pre/zero 오프셋 20 µs, P폭 값 바꾸면 추종
LOOPD, 셀(0,0)	51 / 39	N1 5000 µs 단일 펄스, N2는 계속 Low	D폭 추종, N2 절대 안 올라감
LOOPP, 셀(1,2)	49 = col2 / 38 = row1	같은 파형이 다른 핀에서	핀 규칙: col c → 51−c, row r → 39−r
LOOPR, row 0	25 = RWL, 31 = CK, 53 = CON, 52 = CR	CON Low·CR High ≈100 ms / RWL 100 µs 주기 펄스 ×1000 + CK 토글 / 끝에 CON High → CR Low 3 ms	read_time 100으로 바꾸면 10 ms

설정

- USB만 연결, GUI '시뮬레이션' 해제, 'PSU 제어 사용' 해제, 셀 선택 후 시나리오 LOOP* → ▶ 시작, ■ 중지로 종료
- 프로브 GND = Due GND. 3.3 V 로직. 스코프 Normal 트리거, 소스 = 51번 상승엣지
- 반복 간격 = 'Read 전 대기 [ms]' (기본 100 ms)

판정 / 틀릴 때

- 폭·오프셋이 GUI 값과 ±1 µs 이내, 셀 좌표 ↔ 핀 일치
- 핀이 다르면 .ino 상단 N1_PIN / N2_PIN / WL_PIN / ADC_CH 테이블만 수정 후 재업로드
- 펄스가 아예 없으면: 펌웨어 Rev2 업로드 여부, COM 포트, 115200 baud 확인

2·3단계: 보드 + PSU, 그리고 소자 삽입 후

단계	조건	프로브	기대 / 확인
2 — 보드+PSU (소자 없음)	PSU 제어 ON, 안전값: WWL_ON +1, WWL_OFF −0.5, WBL_PROG +1, WBL_OFF 0, VREAD −0.5, VDD 5	S1 mux 출력(WTR gate), BLTOP mux 출력(WBL)	LOOPP: WTR gate가 WWL_OFF↔WWL_ON, WBL이 WBL_OFF↔WBL_PROG로 Arduino와 같은 타이밍에 스윙. 지연·에지가 pre/zero(20 µs)보다 크면 값 늘림
	'전압 적용' 후 멀티미터	각 mux 입력	PSU 설정값·극성 일치(반전 rail은 음수). GUI '측정값 읽기'와 비교
	LOOPR	RWL(read TR gate), 적분기 V_OUT	RWL이 VINP↔VREAD 오감. 소자 없으니 V_OUT은 CR High 동안 평탄(누설만)
3 — 소자 삽입	INIT → DBGR	GUI 맵	25셀 값이 0/1023에 붙지 않게 VREAD·read_time 조정
	LOOPR	V_OUT	RWL High 100 ms 동안 선형 상승 → CON High에서 정지 → CR Low 리셋. 3.3 V 근처면 read_time ↓
	DBGP 1회 → DBGR, DBGD 1회 → DBGR	GUI 맵	선택 셀만 변하고 나머지 Δ≈0. 이후 PD(phase_voltages)로 본 측정

알아 두어야 할 것

- 모든 PSU·Arduino GND 한 점 공통. 반전 rail(WWL_OFF, WBL_OFF, VREAD)은 + 단자를 GND에. 보드 전원 PSU3는 먼저 ON, 마지막에 OFF
- VREAD는 VINP(가상접지)보다 낮아야 ADC가 올라감 — 0 → −0.5 → −1 V로 내리며 ADC 상승을 보면 극성이 확인됨. VINP 값은 보드에서 확인
- LOOP* 중 GUI 그래프는 비어 있는 게 정상. 측정 데이터(data/)는 git에 올라가지 않음

요약

전압은 PSU rail(GUI 전압표/yaml voltages), 타이밍·횟수·셀은 GUI/yaml, 핀·주기는 펌웨어.
WBL_OFF는 P phase 0 V / D phase −3 V로 나눠 쓰고(phase_voltages), VREAD는 음수.
소자 전엔 LOOPP/LOOPD/LOOPR로 핀 → mux 출력 → 적분기 순으로 파형을 확인한다.